TRƯỜNG ĐẠI HỌC CỬU LONG • KHOA CNTT – TRÍ TUỆ NHÂN TẠO

BÀI GIẢNG KIỂM THỬ PHẦN MỀM

CHƯƠNG 4 – BUỔI 3

KIỂM THỬ TĨNH (STATIC TESTING)

Tổng quan kỹ thuật kiểm thử • Quy trình xem xét (review) • Phân tích mã nguồn

Thời lượng: **5 tiết** (1 tiết = 45 phút)

Biên soạn: ThS. Lôi Thị Tú Trân

Vĩnh Long – 2026

Mục tiêu buổi học

Kiến thức

Phân biệt được kỹ thuật, loại, cấp độ, phương thức và công cụ kiểm thử.

Kiến thức

So sánh được kiểm thử tĩnh và kiểm thử động về mục đích, đối tượng và thời điểm.

Kiến thức

Trình bày được quy trình xem xét và bốn hình thức review theo ISTQB.

Kỹ năng

Phát hiện được khiếm khuyết trong tài liệu yêu cầu và viết lại yêu cầu cho kiểm thử được.

Kỹ năng

Phân tích được một đoạn mã nguồn: chỉ ra vấn đề công cụ bắt được và vấn đề cần con người.

Thái độ

Nhận thức kiểm thử tĩnh là cách phát hiện khiếm khuyết **sớm nhất và rẻ nhất**.

Tương ứng **Buổi 3 – 5 tiết** trong Đề cương chi tiết học phần. Chuẩn tham chiếu: **ISTQB CTFL v4.0** – Chapter 3: Static Testing.

Nội dung buổi học

- **4.1** Tổng quan về kỹ thuật kiểm thử – phân biệt các khái niệm, căn cứ lựa chọn
- **4.3** Kiểm thử tĩnh (Static Testing)
  - Khái niệm, đặc điểm và vai trò • So sánh tĩnh – động
  - Quy trình xem xét sản phẩm công việc • Năm vai trò trong review
  - Bốn hình thức xem xét • Ví dụ xem xét tài liệu yêu cầu
- **4.4** Kỹ thuật phân tích mã nguồn (Source Code Analysis)
  - Xem xét thủ công và phân tích tĩnh tự động • Các nhóm vấn đề và chỉ số
  - Cảnh báo đúng – cảnh báo sai – vấn đề bị bỏ sót • Ví dụ minh hoạ
- **4.5** Bài tập vận dụng + Bài tập tự luyện + Câu hỏi ôn tập

THỜI LƯỢNG: ≈ 1 TIẾT

4.1

Tổng quan về kỹ thuật kiểm thử

- Khái niệm và vai trò
- Từ cơ sở kiểm thử đến test case
- Phân biệt kỹ thuật – loại – cấp độ
- Căn cứ lựa chọn

Kỹ thuật kiểm thử là gì?

Kỹ thuật kiểm thử (Test Technique) là phương pháp **có hệ thống** dùng trong phân tích và thiết kế kiểm thử, nhằm xác định nội dung cần kiểm tra, xây dựng dữ liệu và hình thành các trường hợp kiểm thử.

CẦN KIỂM THỬ ĐIỀU GÌ?

Xác định **điều kiện kiểm thử** (test conditions) từ cơ sở kiểm thử

KIỂM THỬ BẰNG CÁCH NÀO?

Chọn **phần tử bao phủ** và **dữ liệu** để tạo trường hợp kiểm thử

LỢI ÍCH CỦA VIỆC ÁP DỤNG KỸ THUẬT KIỂM THỬ

- Xây dựng test case **có tổ chức và logic**, thay vì làm theo cảm tính
- **Bao phủ** được các điều kiện và tình huống quan trọng của hệ thống
- **Tối ưu số lượng test case** – tránh dư thừa nhưng vẫn giữ hiệu quả phát hiện lỗi
- Cho phép **đo lường và giải thích** được: đã kiểm tra gì, còn thiếu gì

Từ cơ sở kiểm thử đến đánh giá kết quả

BƯỚC 1

Cơ sở kiểm thử
Test basis

Yêu cầu, tiêu chí chấp nhận, mô hình thiết kế, giao diện, mã nguồn, dữ liệu lỗi lịch sử.

BƯỚC 2

Điều kiện kiểm thử
Test condition

Khía cạnh có thể kiểm tra được. VD: "giới hạn tín chỉ", "điều kiện học phần tiên quyết".

BƯỚC 3

Phần tử bao phủ
Coverage item

Đối tượng cụ thể phải thực hiện: một vùng tương đương, một quy tắc, một trạng thái, một nhánh mã.

BƯỚC 4

Dữ liệu kiểm thử
Test data

Dữ liệu đầu vào và dữ liệu liên quan cần để kích hoạt phần tử bao phủ.

BƯỚC 5

Trường hợp kiểm thử
Test case

Tiền điều kiện, dữ liệu, các bước, **kết quả mong đợi**, hậu điều kiện.

BƯỚC 6

Kết quả thực tế
Actual result

So sánh với kết quả mong đợi để kết luận **Đạt / Không đạt**.

Phân biệt các khái niệm dễ nhầm lẫn trong kiểm thử

Khái niệm	Câu hỏi chính	Ví dụ
Kỹ thuật kiểm thử (Test technique)	Thiết kế trường hợp kiểm thử **bằng cách nào**?	Phân vùng tương đương, giá trị biên, bảng quyết định
Loại kiểm thử (Test type)	Đặc tính hoặc mục tiêu **nào được đánh giá**?	Kiểm thử chức năng, hiệu năng, khả năng sử dụng
Cấp độ kiểm thử (Test level)	Đối tượng được kiểm tra ở **phạm vi nào**?	Đơn vị, tích hợp, hệ thống, chấp nhận
Phương thức thực hiện	Có **cần vận hành** đối tượng kiểm thử không?	Kiểm thử động (có chạy) – kiểm thử tĩnh (không chạy)
Công cụ kiểm thử (Test tool)	Phần mềm nào **hỗ trợ** hoạt động kiểm thử?	Jira, TestRail, Selenium, SonarQube, JMeter

Một kỹ thuật dùng được ở nhiều cấp độ và cho nhiều loại kiểm thử. VD: giá trị biên dùng cho một hàm ở mức đơn vị, cũng dùng cho một biểu mẫu ở mức hệ thống.

Căn cứ lựa chọn kỹ thuật kiểm thử

Căn cứ	Ý nghĩa đối với việc lựa chọn
Mục tiêu kiểm thử	Cần đánh giá hành vi, cấu trúc mã, chất lượng tài liệu hay đặc tính phi chức năng?
Đặc điểm đối tượng	Đối tượng có thực thi được không? Có nhiều trạng thái? Quy tắc nghiệp vụ có phức tạp?
Cơ sở kiểm thử	Hiện có yêu cầu, mô hình, mã nguồn hay chỉ có dữ liệu lỗi lịch sử?
Rủi ro	Ưu tiên kỹ thuật kiểm tra sâu các chức năng hoặc khu vực quan trọng, rủi ro cao.
Mức đầy đủ của đặc tả	Đặc tả rõ ⇒ dùng kỹ thuật hộp đen. Đặc tả thiếu ⇒ bổ sung kỹ thuật dựa trên kinh nghiệm.
Thời gian & nguồn lực	Quyết định độ sâu, số lượng trường hợp và mức chính thức của hoạt động kiểm thử.
Kỹ năng nhóm	Một số kỹ thuật đòi hỏi hiểu nghiệp vụ sâu, biết mô hình hoá hoặc đọc được mã nguồn.
Yêu cầu tuân thủ	Hệ thống quan trọng có thể bắt buộc mức bao phủ và bằng chứng xem xét chặt chẽ hơn.

Ví dụ phối hợp kỹ thuật – chức năng đăng ký học phần

YÊU CẦU: Sinh viên chỉ được đăng ký khi tài khoản đang hoạt động, đã hoàn thành học phần tiên quyết, lớp còn chỗ và tổng tín chỉ sau đăng ký không vượt quá 25 (miền hợp lệ: 1–25).

- **Phân vùng tương đương + Giá trị biên** → kiểm tra miền tín chỉ 1–25
- **Bảng quyết định** → kiểm tra các tổ hợp trạng thái tài khoản, tiên quyết, chỗ trống, giới hạn tín chỉ
- **Chuyển trạng thái** → kiểm tra quá trình: chưa đăng ký → chờ xác nhận → thành công → đã hủy
- **Bao phủ câu lệnh và nhánh** → đánh giá phần mã xử lý điều kiện đăng ký đã được thực thi
- **Xem xét tĩnh** → phát hiện điểm chưa rõ: 25 tín chỉ tính theo học kỳ hay theo từng đợt đăng ký?
- **Đoán lỗi + Thăm dò** → nhấn nút nhiều lần tạo bản ghi trùng, hai sinh viên tranh chỗ cuối cùng

Mỗi kỹ thuật quan sát sản phẩm từ một góc độ ⇒ phải phối hợp nhiều kỹ thuật mới đạt độ tin cậy cần thiết.

THỜI LƯỢNG: ≈ 1,5 TIẾT

4.3

Kỹ thuật kiểm thử tĩnh (Static Testing)

- Khái niệm và vai trò
- Quy trình xem xét
- Vai trò trong review
- Các hình thức xem xét

So sánh kiểm thử tĩnh và kiểm thử động

Tiêu chí	Kiểm thử tĩnh (Static Testing)	Kiểm thử động (Dynamic Testing)
Thực thi đối tượng	**Không** chạy chương trình	**Có** chạy chương trình
Đối tượng	Yêu cầu, thiết kế, mô hình, kế hoạch, test case, tài liệu, mã nguồn	Thành phần hoặc hệ thống **có thể chạy được**
Cách phát hiện	Nhận diện **trực tiếp** bất thường và khiếm khuyết	Quan sát **thất bại** rồi phân tích ngược tìm nguyên nhân
Thời điểm	Có thể bắt đầu **rất sớm**, ngay khi có bản nháp tài liệu	Chỉ khi đã có đối tượng thực thi được
Ví dụ phát hiện	Mơ hồ, mâu thuẫn, thiếu ngoại lệ, mã không thể tiếp cận, biến chưa khởi tạo	Sai kết quả tính toán, lỗi thời gian chạy, rò rỉ bộ nhớ, hiệu năng kém
Giá trị chính	**Phòng ngừa** – cải thiện sản phẩm trước khi nó thành đầu vào bước sau	**Phát hiện** – xác nhận hành vi thực tế đúng như mong đợi

Hai phương thức **bổ sung cho nhau, không thay thế nhau**. Kiểm thử tĩnh phát hiện được những khiếm khuyết mà kiểm thử động không bao giờ thấy (và ngược lại).

Đối tượng và khiếm khuyết có thể phát hiện bằng kiểm thử tĩnh

Sản phẩm công việc	Ví dụ khiếm khuyết có thể phát hiện
Tài liệu yêu cầu	Mơ hồ, mâu thuẫn giữa các mục, thiếu xử lý ngoại lệ, yêu cầu **không thể kiểm thử được**, thuật ngữ không nhất quán.
Thiết kế và mô hình	Thiếu thành phần hoặc luồng xử lý, giao diện giữa các module không thống nhất, thiếu cơ chế xử lý lỗi.
Kế hoạch, test case, dữ liệu	Không truy vết được về yêu cầu, thiếu kết quả mong đợi, dữ liệu không đủ, bước thực hiện không rõ ràng.
Mã nguồn	Biến chưa khởi tạo, mã không thể tiếp cận, mã trùng lặp, độ phức tạp quá cao, vi phạm quy tắc viết mã.

Quy trình xem xét sản phẩm công việc (Work Product Review Process)

BƯỚC 1

Lập kế hoạch Planning

Xác định mục tiêu, phạm vi, sản phẩm cần xem xét, tiêu chí, thành viên tham gia và cách ghi nhận.

BƯỚC 2

Khởi động Initiate review

Cung cấp **đúng phiên bản** tài liệu, mục tiêu, phân công vai trò và tài liệu tham chiếu.

BƯỚC 3

Xem xét cá nhân Individual review

Mỗi thành viên **tự đọc**, nhận diện bất thường và ghi nhận **trước khi họp** – đây là bước tìm ra nhiều lỗi nhất.

BƯỚC 4

Trao đổi & phân tích Communication

Phân loại bất thường thành: khiếm khuyết, câu hỏi, đề xuất cải tiến hoặc nội dung ngoài phạm vi.

BƯỚC 5

Sửa chữa & theo dõi Fixing & reporting

Tác giả sửa, người phụ trách xác minh và kết thúc hoặc mở vòng xem xét mới.

Năm vai trò cơ bản trong hoạt động xem xét

Vai trò	Trách nhiệm chính
Quản lý (Manager)	Quyết định nội dung cần xem xét, phân bổ nguồn lực và thời gian.
Tác giả (Author)	Tạo ra sản phẩm, giải thích khi cần và **thực hiện sửa chữa** sau khi xem xét.
Người điều phối (Facilitator / Moderator)	Tổ chức hoạt động, bảo đảm đạt mục tiêu, kiểm soát thời gian và giữ **trao đổi khách quan**.
Người xem xét (Reviewer)	Đọc sản phẩm, nhận diện bất thường và đưa ra nhận xét chuyên môn.
Người ghi chép (Scribe)	Ghi lại bất thường, quyết định, người phụ trách xử lý và thời hạn.

Trong hoạt động nhỏ, **một người có thể đảm nhiệm nhiều vai trò**. Với xem xét chính thức, việc phân tách vai trò giúp tăng tính khách quan và khả năng theo dõi.

So sánh bốn hình thức xem xét (Review Types)

Tiêu chí	Informal Review (Không chính thức)	Walkthrough (Tuần tự)	Technical Review (Kỹ thuật)	Inspection (Nghiêm ngặt)
Mức chính thức	Thấp	Thấp – Trung bình	Trung bình – Cao	**Cao nhất**
Người dẫn dắt	Không cố định	**Tác giả**	Người điều phối	Người điều phối được đào tạo
Chuẩn bị cá nhân	Không bắt buộc	Có thể có	Thường có	**Bắt buộc**
Mục tiêu nổi bật	Phản hồi nhanh	Hiểu & kiểm tra nội dung, chia sẻ kiến thức	Đánh giá & đạt đồng thuận kỹ thuật	Phát hiện **tối đa** bất thường
Ghi nhận	Đơn giản	Có thể ghi biên bản	Có báo cáo chính thức	Có báo cáo **và số liệu đo lường**
Phù hợp với	Sản phẩm nhỏ, rủi ro thấp	Chia sẻ kiến thức trong nhóm	Vấn đề kỹ thuật quan trọng	Sản phẩm quan trọng, cần kiểm soát chặt

Ví dụ xem xét tài liệu yêu cầu

YÊU CẦU BAN ĐẦU: "Sinh viên được đăng ký tối đa 25 tín chỉ và hệ thống phải xử lý nhanh."

CÂU HỎI NHÓM XEM XÉT CẦN ĐẶT RA

- 25 tín chỉ tính theo **học kỳ** hay theo **đợt đăng ký**?
- Giá trị **đúng 25** có được chấp nhận không (≤ hay <)?
- Học phần **đang chờ xác nhận** có được tính vào tổng không?
- **"Xử lý nhanh"** là bao nhiêu giây, ở điều kiện tải nào?
- Điều gì xảy ra khi **hai sinh viên cùng đăng ký chỗ cuối cùng**?
- Sinh viên diện đặc biệt (học vượt) có được vượt 25 không?

YÊU CẦU SAU KHI VIẾT LẠI

"Trong thời gian đăng ký, hệ thống **chỉ chấp nhận yêu cầu nếu tổng số tín chỉ của các lớp đã đăng ký thành công và lớp đang yêu cầu đăng ký trong học kỳ không vượt quá 25**.

Hệ thống phải phản hồi kết quả **trong tối đa ba giây** ở điều kiện tải quy định trong yêu cầu hiệu năng."

Giá trị của kiểm thử tĩnh: một buổi review 60 phút đã loại bỏ được ít nhất 3 khiếm khuyết yêu cầu – nếu để lọt tới giai đoạn vận hành, chi phí sửa sẽ gấp hàng chục lần.

THỜI LƯỢNG: ≈ 1 TIẾT

4.4

Kỹ thuật phân tích mã nguồn (Source Code Analysis)

- Thủ công vs tự động
- Các nhóm vấn đề & chỉ số
- Cảnh báo đúng – sai – bỏ sót

Xem xét mã thủ công và Phân tích tĩnh tự động

MANUAL CODE REVIEW – Thủ công

- Con người **trực tiếp đọc và đánh giá** mã nguồn
- **Phát hiện tốt:** sai quy tắc nghiệp vụ, thuật toán không phù hợp, tên biến gây hiểu sai, xử lý ngoại lệ chưa đúng yêu cầu, cấu trúc khó bảo trì
- **Hạn chế:** chậm, tốn nhân lực, khó áp dụng cho khối lượng mã lớn, phụ thuộc kinh nghiệm người xem xét

AUTOMATED STATIC ANALYSIS – Tự động

- Công cụ kiểm tra mã **mà không thực thi chương trình**
- **Phân tích được:** cú pháp, luồng điều khiển, luồng dữ liệu, độ phức tạp, mã trùng lặp, mẫu vấn đề đã biết
- **Ưu điểm:** tốc độ, tính lặp lại, xử lý khối lượng lớn
- **Hạn chế:** không hiểu **mục tiêu nghiệp vụ** và bối cảnh thiết kế; dễ tạo cảnh báo sai

SAST (Static Application Security Testing) là hướng ứng dụng tập trung vào **điểm yếu bảo mật**. SAST **không đồng nghĩa** với toàn bộ phân tích tĩnh mã nguồn – phân tích tĩnh còn phục vụ khả năng bảo trì, chất lượng và tuân thủ quy tắc.

Các nhóm vấn đề có thể phát hiện khi phân tích mã nguồn

Nhóm vấn đề	Ví dụ	Phương thức phù hợp
Mã không sử dụng	Biến, hàm hoặc đoạn mã không bao giờ được gọi	Công cụ và xem xét
Mã không thể tiếp cận	Câu lệnh nằm sau lệnh `return` vô điều kiện	Công cụ
Biến và dữ liệu	Biến chưa khởi tạo, nguy cơ tham chiếu null	Công cụ và xem xét
Luồng điều khiển	Điều kiện luôn đúng, nhánh bất hợp lý, vòng lặp vô hạn	Công cụ và xem xét
Mã trùng lặp	Nhiều đoạn xử lý giống hệt nhau (copy-paste)	Công cụ
Độ phức tạp	Hàm có quá nhiều điểm quyết định và nhiều trách nhiệm	Công cụ và xem xét
Quy tắc viết mã	Tên biến, định dạng, quy ước không thống nhất	Công cụ
Xử lý ngoại lệ	Bỏ qua lỗi (catch rỗng) hoặc bắt ngoại lệ quá rộng	Xem xét và công cụ
Bảo mật	Luồng dữ liệu không an toàn, mật khẩu/khoá bí mật ghi trực tiếp trong mã	Công cụ và chuyên gia
Sai nghiệp vụ	Thiếu điều kiện tiên quyết, tính sai giới hạn, sai quy tắc tính toán	**Chủ yếu xem xét thủ công**

Độ phức tạp chu trình (Cyclomatic Complexity) phản ánh số đường đi độc lập tuyến tính. Hàm nhiều điểm quyết định ⇒ khó hiểu, cần nhiều test case hơn. Đây là **tín hiệu để xem xét**, không phải kết luận duy nhất về chất lượng mã.

Quy trình phân tích mã nguồn

BƯỚC 1

Xác định mục tiêu & phạm vi

Phân tích để tìm lỗi bảo mật, cải thiện bảo trì hay kiểm tra tuân thủ quy tắc?

BƯỚC 2

Chọn bộ quy tắc & cấu hình

Chọn coding standard, mức nghiêm ngặt, loại trừ thư viện bên thứ ba.

BƯỚC 3

Chạy công cụ / xem xét thủ công

Thực hiện phân tích và thu thập toàn bộ cảnh báo.

BƯỚC 4

Phân loại & xác minh cảnh báo

Sắp theo loại và mức ưu tiên; **xác minh trong bối cảnh thực tế** để loại cảnh báo sai.

BƯỚC 5

Sửa mã & phân tích lại

Sửa hoặc ghi nhận lý do chấp nhận ngoại lệ; chạy lại để kiểm tra kết quả sửa; theo dõi xu hướng qua các phiên bản.

Cảnh báo đúng – Cảnh báo sai – Vấn đề bị bỏ sót

Cảnh báo đúng (True positive)

Công cụ nhận diện **vấn đề thực sự tồn tại** và có ý nghĩa trong bối cảnh mã nguồn. Cần được sửa hoặc ghi nhận.

Cảnh báo sai (False positive)

Công cụ báo vấn đề nhưng sau khi xác minh, mã **không gây khiếm khuyết** trong bối cảnh thực tế. Quá nhiều cảnh báo sai làm nhóm mất niềm tin vào công cụ.

Vấn đề bị bỏ sót (False negative)

Mã **có vấn đề nhưng công cụ không phát hiện** vì thiếu quy tắc, thiếu thông tin hoặc luồng quá phức tạp. **Không có cảnh báo ≠ không có khiếm khuyết.**

⇒ Kết quả công cụ **phải được con người xác minh** và phối hợp với xem xét mã cùng kiểm thử động.

Ví dụ phân tích mã nguồn – hàm kiểm tra điều kiện đăng ký


```
public bool CanRegister(Student student, Course course, int currentCredits)
{
    bool isValid = true;                        // (1) khai bao nhung khong dung
    if (student.Status == "Locked")             // (2) chuoi viet truc tiep
        return false;
    if (course.AvailableSeats < 0)              // (3) dieu kien dang ngo
        return false;
    int totalCredits = currentCredits + course.Credits;
    if (totalCredits > 25)                      // (4) so 25 viet truc tiep
        return false;
    return true;
}
```

Kết quả phân tích đoạn mã trên

CÔNG CỤ CÓ THỂ CẢNH BÁO

- Biến `isValid` được **khai báo nhưng không sử dụng**
- `student` hoặc `course` có **nguy cơ null** (tuỳ cấu hình nullable và bộ quy tắc)
- Chuỗi `"Locked"` và số `25` được **viết trực tiếp trong mã** (magic string / magic number)
- Điều kiện `AvailableSeats < 0` **đáng ngờ** – lớp có 0 chỗ vẫn được chấp nhận

NGƯỜI XEM XÉT PHẢI PHÁT HIỆN THÊM

- Hàm **chưa kiểm tra học phần tiên quyết** – thiếu hẳn một quy tắc nghiệp vụ
- Quy tắc tín chỉ có thể cần **tính cả học phần đang chờ xác nhận**
- Cần xác định **các trạng thái tài khoản khác** ngoài "Locked" (Suspended, Pending...)
- Cần làm rõ hành vi khi **hai yêu cầu tranh chấp chỗ cuối cùng** (race condition)

Bài học: công cụ phù hợp với mẫu vấn đề đã xác định trước, còn **con người phải đánh giá ý nghĩa nghiệp vụ**. Sau khi sửa mã, nhóm phải **phân tích lại và thực hiện kiểm thử động** để xác nhận hành vi.

Ưu điểm, hạn chế và lỗi thường gặp khi phân tích mã nguồn

ƯU ĐIỂM

- Phát hiện **rất sớm**, ngay khi mã vừa được viết; kiểm tra nhanh và lặp lại được; hỗ trợ chuẩn hoá chất lượng mã trong toàn nhóm

HẠN CHẾ

- Phụ thuộc ngôn ngữ, bộ quy tắc và cấu hình; có thể tạo cảnh báo sai hoặc bỏ sót; **không hiểu đầy đủ nghiệp vụ**

LỖI ÁP DỤNG THƯỜNG GẶP

- Chạy công cụ với **cấu hình mặc định** rồi chấp nhận mọi cảnh báo mà không xác minh
- Chỉ quan tâm **số lượng** cảnh báo, bỏ qua **mức ảnh hưởng** của từng vấn đề
- **Không phân tích lại** sau khi sửa mã ⇒ không biết đã sửa đúng hay chưa
- Xem **"không có cảnh báo" là "không có lỗi"** – sai lầm nghiêm trọng nhất

THỜI LƯỢNG: ≈ 1,5 TIẾT

4.5

Bài tập vận dụng và câu hỏi ôn tập

- 2 bài tập
- 10 bài tập tự luyện
- 6 câu hỏi ôn tập

Lựa chọn kỹ thuật kiểm thử phù hợp

- Với mỗi tình huống sau, hãy xác định kỹ thuật kiểm thử phù hợp (**Kiểm thử động / Kiểm thử tĩnh / Phân tích mã nguồn**) và giải thích lý do:
  - **1.** Kiểm tra tài liệu đặc tả yêu cầu trước khi lập trình.
  - **2.** Thực hiện kiểm tra chức năng đăng nhập bằng nhiều bộ dữ liệu khác nhau.
  - **3.** Sử dụng công cụ để phát hiện đoạn mã không được sử dụng trong chương trình.
  - **4.** Kiểm tra hiệu năng của hệ thống khi có nhiều người dùng truy cập đồng thời.
  - **5.** Tổ chức cuộc họp để cùng xem xét tài liệu thiết kế hệ thống.
- **Mở rộng:** với mỗi tình huống, nêu thêm **kỹ thuật thiết kế test cụ thể** hoặc **công cụ** nên dùng.

Phân tích mã nguồn và đề xuất sửa lỗi

- Xét đoạn mã tính chiết khấu sau:
  - `public double calculateDiscount(double total) {`
  - `    if (total > 1000000)        return total * 0.1;`
  - `    else if (total > 1000000)   return total * 0.05;`
  - `    else                        return 0;`
  - `}`
- **Yêu cầu:**
  - a) Đọc và phân tích đoạn mã; xác định các lỗi hoặc điểm bất hợp lý.
  - b) Đề xuất cách sửa để chương trình hoạt động đúng.
  - c) Cho biết các lỗi trên có thể phát hiện bằng **phân tích mã nguồn** hay phải **chạy chương trình** mới phát hiện được?

Bộ 10 bài tập tự luyện – Kiểm thử tĩnh

- **Bài 1.** So sánh kiểm thử tĩnh và kiểm thử động theo 4 tiêu chí. Cho ví dụ một khiếm khuyết mà mỗi loại phát hiện được còn loại kia thì không.
- **Bài 2.** Cho yêu cầu: *"Hệ thống phải xử lý nhanh và thân thiện với người dùng."* Chỉ ra các khiếm khuyết và viết lại cho **kiểm thử được**.
- **Bài 3.** Nêu 5 hoạt động trong quy trình xem xét. Theo bạn hoạt động nào phát hiện được nhiều khiếm khuyết nhất? Giải thích.
- **Bài 4.** So sánh **Walkthrough** và **Inspection** theo 5 tiêu chí. Chọn hình thức phù hợp cho: (a) review user story trong sprint; (b) review thiết kế module thanh toán.
- **Bài 5.** Liệt kê 5 vai trò trong hoạt động xem xét và trách nhiệm của từng vai trò. Điều gì xảy ra nếu **tác giả đồng thời là người điều phối**?
- **Bài 6.** Cho đoạn mã có: biến khai báo không dùng, câu lệnh sau `return`, khối `catch` rỗng, mật khẩu viết thẳng trong mã. Vấn đề nào **công cụ** bắt được, vấn đề nào cần **con người**?
- **Bài 7.** Giải thích **cảnh báo đúng / cảnh báo sai / vấn đề bị bỏ sót** trong phân tích tĩnh. Vì sao *"không có cảnh báo"* không đồng nghĩa *"không có lỗi"*?
- **Bài 8.** Độ phức tạp chu trình là gì? Một hàm có **7 điểm quyết định** cần tối thiểu bao nhiêu test case để phủ hết nhánh? Nêu 2 cách giảm độ phức tạp.
- **Bài 9.** Nhóm chạy SonarQube lần đầu và nhận **1.200 cảnh báo**. Đề xuất quy trình 4 bước để xử lý mà không bị "ngợp" và không bỏ sót vấn đề quan trọng.
- **Bài 10.** Lập **checklist 8 mục** để xem xét tài liệu đặc tả yêu cầu của chức năng "Đăng nhập".

Câu hỏi ôn tập – Buổi 3

1. Trình bày khái niệm **kỹ thuật kiểm thử**. Việc áp dụng kỹ thuật kiểm thử mang lại những lợi ích gì? tr 5
2. So sánh **kiểm thử tĩnh** và **kiểm thử động** về mục đích, phương pháp, thời điểm áp dụng và ưu điểm của mỗi phương thức.tr11
3. Trình bày **năm hoạt động** của quy trình xem xét sản phẩm công việc và **năm vai trò** cơ bản trong hoạt động xem xét.tr13/tr14
4. Phân biệt bốn hình thức xem xét: **Informal Review, Walkthrough, Technical Review, Inspection**.tr15
5. Trình bày khái niệm **phân tích mã nguồn**. Phân biệt phân tích thủ công và tự động; nêu ưu điểm của từng phương pháp.tr18
6. Vì sao trong một dự án nên **kết hợp** kiểm thử tĩnh, kiểm thử động và phân tích mã nguồn thay vì chỉ dùng một kỹ thuật?

Tóm tắt buổi 3 – Kiểm thử tĩnh

- ✓ Kỹ thuật kiểm thử chuyển **cơ sở kiểm thử → điều kiện kiểm thử → phần tử bao phủ → dữ liệu → trường hợp kiểm thử** một cách có hệ thống.
- ✓ **Kiểm thử tĩnh không thực thi chương trình** – nó nhận diện khiếm khuyết **trực tiếp** trong tài liệu và mã nguồn, thay vì quan sát thất bại rồi truy ngược.
- ✓ Giá trị lớn nhất của kiểm thử tĩnh là **phòng ngừa**: sửa khiếm khuyết ngay trong tài liệu, trước khi nó trở thành đầu vào của bước sau.
- ✓ Trong quy trình review, **xem xét cá nhân** là bước phát hiện nhiều khiếm khuyết nhất – đây là lý do Inspection bắt buộc chuẩn bị cá nhân.
- ✓ Công cụ phân tích tĩnh xử lý **mẫu vấn đề đã biết**; **con người** đánh giá ý nghĩa nghiệp vụ. **Không có cảnh báo ≠ không có lỗi.**
- ✓ Kiểm thử tĩnh và kiểm thử động **bổ sung cho nhau, không thay thế nhau** – mỗi loại phát hiện được nhóm khiếm khuyết mà loại kia không thấy.

Đánh giá kết quả học tập – Kỹ thuật kiểm thử tĩnh

BÀI QUIZ • 30%

- **15 câu trắc nghiệm**, thời gian 15 phút
- Làm vào **đầu buổi chiều** ngày 1, 2, 3
- Mỗi câu **0,67 điểm** – làm tròn thang 10
- Chấm chéo tại lớp, công bố đáp án ngay
- Lấy **trung bình 3 bài quiz**

BÀI KIỂM TRA VIẾT 45 PHÚT • 70%

- **20 câu trắc nghiệm + 1 bài tập vận dụng**
- Làm vào **đầu ngày 4**, phủ cả 6 chương
- Trắc nghiệm **5 điểm** – bài tập **5 điểm**
- Mỗi chương góp khoảng **3–4 câu** trắc nghiệm
- Bài tập vận dụng chọn **1 trong 6 đề** của các chương

CÁCH TÍNH ĐIỂM QUÁ TRÌNH

Điểm quá trình = **0,7 × điểm bài kiểm tra viết 45 phút** + **0,3 × trung bình 3 bài quiz**

Ví dụ: bài viết 8,0 và ba quiz lần lượt 7,0 – 8,0 – 9,0 (trung bình 8,0) → điểm quá trình = 0,7 × 8,0 + 0,3 × 8,0 = **8,0**

Quiz 15 câu – Buổi 3 (phần 1/3)

Câu 1. Kiểm thử tĩnh trực tiếp tìm ra:
A. Failure B. Defect C. Error của người dùng D. Rủi ro dự án

Câu 2. Đặc điểm phân biệt của hình thức Walkthrough là:tr15
A. Có checklist bắt buộc B. Tác giả dẫn dắt buổi họp C. Trang trọng nhất D. Không cần chuẩn bị trước

Câu 3. Hình thức xem xét TRANG TRỌNG NHẤT, có vai trò rõ ràng và tiêu chí vào/ra là:tr15
A. Informal review B. Walkthrough C. Technical review D. Inspection

Câu 4. Vai trò nào KHÔNG nên do tác giả đảm nhận trong buổi review?tr15
A. Người xem xét B. Người điều phối C. Thư ký D. Người tham dự

Câu 5. Bước nào quyết định chất lượng của một buổi review?
A. Họp thật lâu B. Chuẩn bị cá nhân – đọc tài liệu trước C. Mời thật nhiều người D. Ghi biên bản thật dài

Quiz 15 câu – Buổi 3 (phần 2/3)

Câu 6. Yêu cầu "hệ thống phải phản hồi nhanh" mắc lỗi gì?
A. Mâu thuẫn B. Thiếu **C**. Không kiểm chứng được D. Trùng lặptr19

Câu 7. Mục 3.1 ghi tối đa 25 tín chỉ, mục 5.4 ghi 30 tín chỉ. Đây là lỗi:
A. Mơ hồ B. Mâu thuẫn C. Thiếu **D.** Không kiểm chứng được

Câu 8. Trong buổi review, hai người tranh cãi cách sửa một vấn đề. Người điều phối nên:
A. Để tranh luận tới khi xong **B.** Ghi nhận vấn đề, hẹn trao đổi cách sửa sau buổi họp___C. Quyết định thay họ D. Kết thúc buổi họp

Câu 9. Công cụ phân tích tĩnh tự động KHÔNG phát hiện được:tr11
A. Biến chưa khởi tạo B. Code chết C. Sai công thức tính học phí D. Rò rỉ tài nguyên

Câu 10. "Cảnh báo sai" (false positive) nghĩa là:tr21
A. Có lỗi nhưng công cụ không báo **B.** Công cụ báo lỗi nhưng thực tế không phải lỗi C. Công cụ báo đúng lỗi D. Công cụ bị treo

Quiz 15 câu – Buổi 3 (phần 3/3)

Câu 11. Quá nhiều cảnh báo sai gây hậu quả nguy hiểm nhất là:
A. Tốn dung lượng ổ đĩa B. Lập trình viên bỏ qua mọi cảnh báo kể cả cảnh báo đúng C. Công cụ chạy chậm D. Không xuất được báo cáo

Câu 12. Đoạn mã: if (x > 10) {...} else if (x > 20) {...}. Vấn đề là:
A. Thiếu dấu ngoặc B. Nhánh thứ hai không bao giờ được chạy tới C. Sai kiểu dữ liệu D. Không có lỗi

Câu 13. Chạy công cụ phân tích tĩnh lần đầu trên dự án cũ ra 3.500 cảnh báo. Nên:
A. Sửa hết ngay trong tuần B. Tắt công cụ đi C. Lọc theo mức nghiêm trọng, xử lý nhóm cao trước, code mới không sinh thêm cảnh báo D. Bỏ qua toàn bộ

Câu 14. Sản phẩm nào sau đây KHÔNG thể áp dụng kiểm thử tĩnh?
A. Đặc tả yêu cầu B. Mã nguồn C. Test case D. Không có – tất cả đều áp dụng được

Câu 15. Ưu điểm lớn nhất của kiểm thử tĩnh so với kiểm thử động là:
A. Không cần người B. Tìm được lỗi rất sớm, khi chưa cần có mã nguồn chạy được C. Luôn tìm được nhiều lỗi hơn D. Thay thế được kiểm thử động

Ngân hàng câu trắc nghiệm cho bài kiểm tra 45 phút – Buổi 3

Câu 1. Một đoạn mã chứa lỗi nằm trong nhánh chưa bao giờ được thực thi trong mọi test case. Kỹ thuật nào CÓ THỂ này?
A. Kiểm thử hộp đen B. Kiểm thử tĩnh (review hoặc phân tích mã nguồn) C. Kiểm thử tải D. Kiểm thử chấp nhận

Câu 2. Nhóm tổ chức buổi review 2 tiếng, 5 người, nhưng không ai đọc tài liệu trước. Hệ quả có khả năng xảy ra nhất:
A. Tìm ra nhiều lỗi logic sâu B. Chỉ nhặt được lỗi chính tả và trình. bày **C** Buổi họp kết thúc sớm D. Không cần biên bản

Câu 3. Chọn hình thức xem xét phù hợp: "Trước khi nộp hồ sơ kiểm định thiết bị y tế, phải rà soát đặc tả theo checklist, có biên bản và chữ ký."tr15
A. Informal review B. Walkthrough C. Technical review D. Inspection

Câu 4. Phát biểu nào ĐÚNG về quan hệ giữa công cụ phân tích tĩnh và review của con người?
A. Công cụ thay thế hoàn toàn con người B. Con người thay thế hoàn toàn công cụ **C.** Công cụ bắt lỗi máy móc, con người bắt lỗi nghiệp vụ – bổ sung cho nhau D. Hai cái tìm cùng một tập lỗi

Xem xét đặc tả và phân tích mã nguồn

Phần 1 – Xem xét đặc tả (2,5 đ). Cho đoạn đặc tả sau:

"Sinh viên được đăng ký học phần trong thời gian quy định. Sinh viên năm cuối được ưu tiên đăng ký trước. Hệ thống phải phản hồi nhanh."

Hãy chỉ ra **ít nhất 4 vấn đề**, mỗi vấn đề nêu rõ thuộc loại nào (**mơ hồ / mâu thuẫn / thiếu / không kiểm chứng được**) và **đề xuất cách viết lại**.

Phần 2 – Phân tích mã nguồn (2,5 đ). Cho hàm tính học phí:

```
double tinhHocPhi(int soTinChi, String loai) {
    double don = 500000;
    if (soTinChi > 25) return -1;
    if (loai == "hocLai") don = don * 1.5;
    return soTinChi * don;
}
```

Hãy chỉ ra **ít nhất 4 vấn đề** trong đoạn mã, phân loại **lỗi logic** hay **vấn đề chất lượng mã**, và **đề xuất cách sửa cụ thể**.